

ICT2142 E-BUSINESS SYSTEMS

Week 05 Practical: Shopping Cart Functionality (Phase 2 Core Development)

Step 4: Project Diary & Technical Implementation Documentation

Course & Module: ICT2142 – E-Business Systems (Phase 2 Core Frontend & Backend)

Project Title: KRONO Watches – Haute Horlogerie Luxury E-Commerce Web Application

Architecture: Next.js 16 (App Router), TypeScript, React Context API, SessionStorage, LocalStorage

GitHub Repository: https://github.com/mohamedrazim11-maker/Krono_Watches.git (main, develop)

1. STATE ARCHITECTURE NOTES

In the Krono Watches application, the shopping cart architecture transitioned from isolated local component state into a centralized React Context Provider (CartProvider in lib/CartContext.tsx) paired with multi-layer browser persistence (SessionStorage and LocalStorage). This guarantees complete reactivity, zero state loss across route navigation, and synchronized visual feedback across all pages.

1.1 Data Flow Lifecycle (From Button Click to Cart State):

- 1. Event Capture:** A user clicks 'Add to Cart' on any storefront surface (Boutique Showcase, Curated Catalogue, Product Detail Page, Quick View Modal, or Wishlist Drawer).
- 2. Payload Normalization:** The addToCart handler receives the product object and extracts unique identifiers: ProductID (id), Title (name), Unit Price (price in LKR), Image (image_url), Brand, and Category.
- 3. Non-Duplication & Accumulation:** The state reducer queries the existing cart array using `findIndex((item) => item.product.id === product.id)`. If the timepiece already exists, its quantity is incremented (`item.quantity + safeQuantity`) without adding duplicate rows. If absent, a new CartItem object is appended.
- 4. Reactive Badge Computation:** Total cart item quantity across the global Navbar is computed dynamically using a memoized reducer (`cart.reduce((sum, item) => sum + item.quantity, 0)`). Any addition, increment, or decrement instantly updates the badge.
- 5. Storage Persistence:** Upon every cart state change, an automated useEffect converts the cart array into a JSON string, synchronously saving it to both sessionStorage and localStorage ('krono_cart'), and broadcasts a sync event.

2. CODE SNIPPETS & LOGIC USED

2.1 Add to Cart Logic with Existing Item Check (Step 1):

```
// lib/CartContext.tsx - The "Add to Cart" Logic
const addToCart = useCallback((product: Product, quantity = 1) => {
  if (!product || !product.id) return;
  const safeQuantity = Math.max(1, quantity);
```

2.2 setCart((prevCart) => {Quantity Up/Down with Edge Case Rule (Step 2):

```
// Check if the item already exists in the cart array

// lib/CartContext.tsx - "Remove Item" using Array filter()
const removeFromCart = useCallback((productId: string) => {
  setCart((prevCart) => prevCart.filter((item) => item.product.id !== productId));
  showToast("Item removed from Cart");
}, [showToast]);

// lib/CartContext.tsx - Quantity Controls with < 1 Edge Case Rule
const updateQuantity = useCallback((productId: string, delta: number) => {
  setCart((prevCart) => {
    prevCart
      .map((item) => {
        if (item.product.id === productId) {
          const newQuantity = item.quantity + delta;
          // Edge Case: If quantity drops below 1, automatically remove item
          if (newQuantity <= 0) return null;
          const safeQuantity = Math.max(1, newQuantity);
          return { ...item, quantity: safeQuantity };
        }
        return item;
      })
      .filter(Boolean) as CartItem[]
  });
}, [showToast]);
```

2.3 Session Storage Persistence & Live Total Calculations (Step 2 & 3):

```
// Step 3: SessionStorage & LocalStorage State Persistence
useEffect(() => {
  if (!hydrated) return;
  try {
    const cartJson = JSON.stringify(cart);
    sessionStorage.setItem("krono_cart", cartJson);
    localStorage.setItem("krono_cart", cartJson);
  } catch (e) {
    console.error("Cart storage sync failed:", e);
  }
}, [cart, hydrated]);

// Step 2: Live Total & Subtotal Calculation (Price x Quantity)
const subtotal = useMemo(() => {
  return cart.reduce((sum, item) => sum + item.product.price * item.quantity, 0);
}, [cart]);
```

3. BUG LOG (TROUBLESHOOTING & STATE SYNCHRONIZATION)

Issue / Bug Description		
<pre>}, [subtotal, couponDiscountPercent]); const grandTotal = useMemo(() => { return Math.max(0, subtotal - discountAmount); }, [subtotal, discountAmount]);</pre>		
Bug 1: Cart Reset On Route Transition	Cart state was stored inside isolated component useState on individual page files (/page.tsx, /catalog/page.tsx). Route transitions caused re-mount and complete state erasure.	Created central CartProvider context in lib/CartContext.tsx, wrapped root layout in app/layout.tsx, and added automatic sessionStorage + localStorage JSON hydration on mount.
Bug 2: Duplicate Product Rows In Cart	Repeatedly clicking 'Add to Cart' appended duplicate objects to the cart array instead of incrementing the quantity of the existing line item.	Implemented findIndex() condition in addToCart to map and increment item.quantity + safeQuantity when matching product.id, appending new elements only if not found.
Bug 3: Stale Badge & Zero Quantity Edge Case	Navbar badge counter counted array rows (cart.length) rather than unit sums. Decrementing items past 1 retained broken 0-quantity rows with broken total price calculations.	Updated badge computation to memoize cart.reduce((sum, i) => sum + i.quantity, 0) and programmed updateQuantity to automatically filter out any item whose quantity drops below 1.

4. GIT COMMIT & REPOSITORY SYNCHRONIZATION

All working cart implementations, persistent state architecture, CTA harmonization, and documentation have been committed, tagged, and pushed to the remote GitHub repository across both develop and main branches.

Commit SHA		
09921d1	main / develop	docs: add Week 05 Step 4 Project Diary and Technical Documentation report
780fd40	main / develop	feat: implement full cart processes with session persistence, quantity controls, filter removal, and update all CTA text to Add to Cart
b4c9bce	main / develop	feat: complete sitewide smooth page transitions and progressive image loading
9c8d303	main / develop	feat: constrain catalog to 5 luxury brands, make responsive with mobile drawer, and add progressive shimmer image loading

Build Verification Status: Next.js 16 Production Build Passed (0 TypeScript / Turbopack errors across all 11 application routes).